Chapter 3: Codebase Tour & File Guide

This chapter walks you through **every file and folder** currently in the repository, explaining what each does and how they connect to one another.

Repository Map

```
Transfinitte_RAG/
├── .gitignore                      <-- Prevents unwanted files from committing to
Git
├── README.md                       <-- High-level project documentation
├── documents/                      <-- This learning guide series
│
├── backend/                        <-- Python FastAPI service
│   ├── .venv/                      <-- Isolated Python virtual environment
│   ├── .env.example                <-- Template for MongoDB & API keys
│   ├── requirements.txt            <-- List of Python packages installed
│   ├── app/
│   │   ├── main.py                 <-- Server entrypoint & route registration
│   │   ├── config.py               <-- Settings & environment variable loader
│   │   ├── core/
│   │   │   └── database.py         <-- MongoDB Atlas connection helper
│   │   ├── models/
│   │   │   └── schemas.py          <-- Data shapes (Citations, Timelines,
Reports)
│   │   ├── api/
│   │   │   ├── routes_ingest.py    <-- Ingestion API endpoints (/api/ingest)
│   │   │   ├── routes_investigate.py<-- Investigation API endpoints
(/api/investigate)
│   │   │   └── routes_eval.py      <-- Evaluation test runner (/api/eval)
│   │   ├── ingestion/
│   │   │   └── chunking.py         <-- Location-preserving chunking logic
│   │   ├── retrieval/
│   │   │   ├── vector_search.py    <-- MongoDB Atlas $vectorSearch queries
│   │   │   └── hybrid_search.py    <-- Hybrid keyword + vector search
│   │   └── reasoning/
│   │       └── investigator.py     <-- LangChain LLM synthesis & abstention
│   ├── synthetic_data/             <-- 5 Multi-source evidence directories
│   │   ├── documents/
│   │   ├── webpages/
│   │   ├── images/
│   │   ├── transcripts/
│   │   └── structured_records/
│   └── tests/
│       └── ground_truth.json       <-- Benchmark ground truth for judge tests
│
└── frontend/                       <-- Next.js 16 Web Dashboard
    ├── package.json                <-- Frontend dependencies & scripts
    ├── tsconfig.json               <-- TypeScript configuration
```

```
    └── src/
        └── app/
            ├── layout.tsx          <-- Root HTML layout
            ├── page.tsx            <-- Main UI dashboard page
            └── globals.css         <-- Styling and design tokens
```

🔍 Detailed File Walkthrough

1. Root Configuration

- `.gitignore`: Tells Git to ignore files that should never be in the cloud repository:
  - `backend/.venv/` (Python virtual environment — hundreds of megabytes of third-party binaries)
  - `frontend/node_modules/` (Node packages)
  - `.env` files (Secret API keys and database passwords)
  - `.next/` (Next.js build cache)

2. Backend Core Files

- **`backend/requirements.txt`**: Specifies every required Python dependency. When you run `pip install -r requirements.txt`, it installs LangChain, FastAPI, PyMongo, PyPDF, etc.
- **`backend/app/main.py`**: The entrypoint of the backend server. It sets up CORS (allowing your Next.js frontend at `localhost:3000` to make requests to `localhost:8000`) and mounts the sub-routers for ingestion, investigation, and evaluation.
- **`backend/app/config.py`**: Uses Pydantic's `BaseSettings` to automatically read configuration values from your `.env` file (like `MONGODB_URI` and `OPENAI_API_KEY`) with strong type checking.
- **`backend/app/core/database.py`**: Creates and manages a reusable connection pool to MongoDB Atlas so the app doesn't open a new database connection on every single request.
- **`backend/app/models/schemas.py`**: Defines the data contracts using Pydantic:
  - `Citation`: Captures the source title, type, exact location (page/timestamp/line), and snippet.
  - `TimelineEvent`: Captures an event on the incident timeline.
  - `InvestigationReport`: The final response containing the timeline, root cause, citations, conflicting evidence, and abstention flag.

3. Backend Retrieval & Reasoning Files

- **`backend/app/ingestion/chunking.py`**: Chunks long documents into smaller segments for embedding, while making sure the page number or timestamp is attached to every single chunk.
- **`backend/app/retrieval/vector_search.py`**: Executes vector queries using MongoDB Atlas's `$vectorSearch` pipeline stage.
- **`backend/app/retrieval/hybrid_search.py`**: Merges semantic vector matches with exact keyword matches using Reciprocal Rank Fusion.
- **`backend/app/reasoning/investigator.py`**: The brain of the system. Uses LangChain to feed the retrieved chunks to the LLM with structured prompts, checking for contradictions and deciding whether to abstain.

4. Synthetic Data & Tests

- `backend/synthetic_data/`: Contains the synthetic incident files across all 5 source types:
  - `documents/`: Runbooks & Architecture specs.
  - `webpages/`: Gateway v2.4.0 release notes.
  - `images/`: Grafana latency dashboard screenshots.
  - `transcripts/`: Incident call logs between engineers.
  - `structured_records/`: Payment microservice JSON logs.
- `backend/tests/ground_truth.json`: The ground-truth reference dataset for the 4 hackathon judge evaluation tests.

5. Frontend Files

- `frontend/package.json`: Manages Next.js, React, and Tailwind dependencies.
- `frontend/src/app/page.tsx`: The main dashboard interface where the user submits incident questions, views the timeline, and inspects citations.